AMERICAN UNIVERSITY OF BEIRUT

COMPARISON OF REINFORCEMENT LEARNING AGENTS FOR THE TASK OF PATH FOLLOWING

by

AHMAD ASSAAD

A Monthly Progress Report Detailing the First Milestone of the Project:
A Comparison Between a Basic Reinforcement Learning Agent and a Physics-Informed Reinforcement Learning Agent.

Submitted as part of the deliverables for VIPP 201A
American University of Beirut
Under the supervision of Professor Naseem Daher.

Contents

1 Introduction **2**

2 Background Information **2**

2.1 Kinematic Bicycle Model 2

2.2 Pure Pursuit Algorithm 2

2.3 Reinforcement Learning 3

2.4 Theoretical Study of Q-Learning Convergence 3

3 Mathematical Formulation of the MDP **4**

3.1 State Space 5

3.2 Action Space 5

3.3 Transition Dynamics 5

3.4 Reward Function 5

3.5 Theoretical Convergence Considerations 6

3.6 Motivation for Alternative Formulations 6

4 Results **7**

4.1 Basic RL Agent 7

4.2 Physics-Informed RL Agent 8

5 Literature Review **10**

6 Conclusion and Future Work **11**

1 Introduction

This report summarizes my work in developing and comparing different RL agents for solving a path-following problem in the racetrack scenario of Highway-Env. The motivation behind this work lies in controlling path-following under varying environmental conditions such as different terrains, varying obstacles, and extreme vehicle dynamics. Under such conditions, standard methods no longer hold, and learning-based methods offer the advantage of adaptability.

In this work, I present a comparison of RL agents' performance under different observation and action spaces, as well as reward functions, attempting to rationalize how and why changing these spaces led to improved performance.

The flow of the report is as follows: I first introduce the necessary background on Pure Pursuit and Reinforcement Learning, then shift toward a brief mathematical analysis of when and how Q-learning converges. Following that, I present a detailed comparison of the different agents and conclude with a brief literature review and discussion of future work.

2 Background Information

2.1 Kinematic Bicycle Model

We model the vehicle using the kinematic bicycle model. The state vector and its derivative are

$$x = \begin{bmatrix} x \\ y \\ \theta \\ v \end{bmatrix}, \qquad \dot{x} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{v} \end{bmatrix}.$$

The governing equations are

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta,$$

$$\dot{\theta} = \omega = \frac{v\tan\delta}{L}, \qquad \dot{v} = a.$$

The model assumes pure rolling, no lateral slip, and low-speed motion.

2.2 Pure Pursuit Algorithm

Pure pursuit is a geometric path-following algorithm.

$$x^2 + y^2 = L_d^2$$

$$\alpha = \tan^{-1}\left(\frac{y_{\text{target}}}{x_{\text{target}}}\right)$$

$$\delta = \tan^{-1}\left(\frac{2L\sin\alpha}{L_d}\right)$$

$$x_{k+1} = x_k + v\cos\psi\,\Delta t$$

$$y_{k+1} = y_k + v\sin\psi\,\Delta t$$

$$\psi_{k+1} = \psi_k + \frac{v}{L}\tan\delta\,\Delta t$$

2.3 Reinforcement Learning

Reinforcement Learning (RL) studies how an agent interacts with an environment in order to maximize cumulative, discounted reward. The problem is formulated as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$V^\pi(s) = E_\pi[G_t \mid S_t = s], \qquad Q^\pi(s,a) = E_\pi[G_t \mid S_t = s, A_t = a]$$

$$Q^*(s,a) = E\left[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid s, a\right]$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\right]$$

2.4 Theoretical Study of Q-Learning Convergence

In this section, we examine the necessary conditions for the convergence of Q-learning.

Consider a finite Markov Decision Process (MDP). Tabular Q-learning converges to the optimal action-value function Q^* with probability one if the following conditions hold:

1. **Finite state and action spaces.** The MDP has finite $\mathcal{S}$ and $\mathcal{A}$.
2. **Sufficient exploration.** Every state–action pair (s, a) is visited infinitely often.
3. **Learning rate conditions.** The step sizes $\alpha_t(s, a)$ satisfy

$$\sum_{t=0}^{\infty} \alpha_t(s,a) = \infty, \qquad \sum_{t=0}^{\infty} \alpha_t^2(s,a) < \infty.$$

A common choice is

$$\alpha_t = \frac{1}{N_t(s,a)}.$$

4. **Bounded rewards.** The reward sequence has bounded variance.

Under these conditions,

$$Q_t(s,a) \xrightarrow{a.s.} Q^*(s,a).$$

The convergence result relies on viewing the Q-learning update as a stochastic approximation of the Bellman optimality operator, which is a contraction mapping under the supremum norm.

Practical Enforcement of Convergence Conditions In practical implementations, strict infinite visitation is approximated through persistent exploration strategies such as ε-greedy policies with slow decay, randomized initial conditions, or injected disturbances to enlarge state-space coverage.

The learning rate condition is typically satisfied using diminishing step sizes, for example

$$\alpha_t(s,a) = \frac{1}{N_t(s,a)}.$$

In contrast, constant learning rates are commonly used in practice for stability and faster learning, but they do not strictly satisfy the stochastic approximation conditions and therefore yield convergence to a neighborhood of Q^* rather than exact asymptotic convergence.

Finally, reward boundedness is ensured through careful reward design or explicit clipping to maintain numerical stability and bounded variance.

3 Mathematical Formulation of the MDP

Let the vehicle state at time t be

$$x_t = (x_t, y_t, \theta_t, v_t).$$

We define the tracking errors relative to the reference path as

$$e_y = \text{lateral error}, \qquad e_\psi = \text{heading error}.$$

In the extended formulation, we additionally define

$$e_\kappa = \kappa_{\text{desired}} - \kappa_{\text{actual}},$$

where κ denotes curvature.

3.1 State Space

The state is defined as

$$s_t = \begin{cases} (e_y, e_\psi) & \text{Basic formulation} \\ (e_y, e_\psi, e_\kappa) & \text{Physics-informed formulation} \end{cases}$$

After discretization, $\mathcal{S}$ is finite.

3.2 Action Space

Initially, the action is the steering angle

$$a_t = \delta_t \in [-\delta_{\max}, \delta_{\max}].$$

In the modified formulation, the action is curvature

$$a_t = \kappa_t, \qquad \delta_t = \tan^{-1}(L\kappa_t).$$

After discretization, $\mathcal{A}$ is finite.

3.3 Transition Dynamics

The transition follows directly from the kinematic bicycle model:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \frac{v\tan\delta}{L}.$$

Given (s_t, a_t), the next state s_{t+1} is uniquely determined under deterministic dynamics. Therefore,

$$P(s_{t+1} \mid s_t, a_t)$$

depends only on (s_t, a_t), satisfying the Markov property.

3.4 Reward Function

The reward is chosen as

$$R_t = -\left(\lambda_y e_y^2 + \lambda_\psi e_\psi^2 + \lambda_\kappa e_\kappa^2\right) + R_{\text{survival}}.$$

Since errors are bounded by track geometry and discretization,

$$|R_t| \leq R_{\max},$$

ensuring bounded rewards.

3.5 Theoretical Convergence Considerations

We now verify the convergence assumptions of tabular Q-learning.

- **Finite MDP.** State and action spaces are discretized; hence $\mathcal{S}$ and $\mathcal{A}$ are finite.
- **Bounded Rewards.** The quadratic reward structure guarantees bounded variance.
- **Markov Property.** State transitions depend only on (s_t, a_t) via deterministic vehicle dynamics.

Thus, the problem satisfies the structural assumptions required for Q-learning convergence in the tabular setting.

3.6 Motivation for Alternative Formulations

In the basic formulation, the state contains only tracking errors. The agent must implicitly infer curvature dynamics from these errors, increasing the complexity of the value function approximation.

By augmenting the state with curvature error e_κ, the transition dynamics become more directly aligned with the control input. In effect, the state representation becomes closer to a sufficient statistic of the system dynamics, potentially improving learning stability and convergence speed.

Furthermore, redefining the action as curvature rather than steering angle aligns the control variable directly with the vehicle's instantaneous curvature:

$$\dot{\theta} = v\kappa.$$

This creates a more linear relationship between action and heading evolution, which may reduce value function nonlinearity and improve numerical stability during learning.

Therefore, while both formulations satisfy the theoretical convergence conditions in the tabular limit, their induced value landscapes differ, leading to different empirical learning behaviors.

4 Results

For each agent, we visualize and analyze three primary metrics: error per step, steps per episode, and the learned policy.

Error per step represents how well on average the agent stays on the desired path. Steps per episode represents survival time, which is the ultimate objective of the task. The learned policy heatmaps ensure that the agent is not reward hacking (i.e., exploiting unintended loopholes in the reward function).

4.1 Basic RL Agent

The figures for the basic RL agent are shown below.

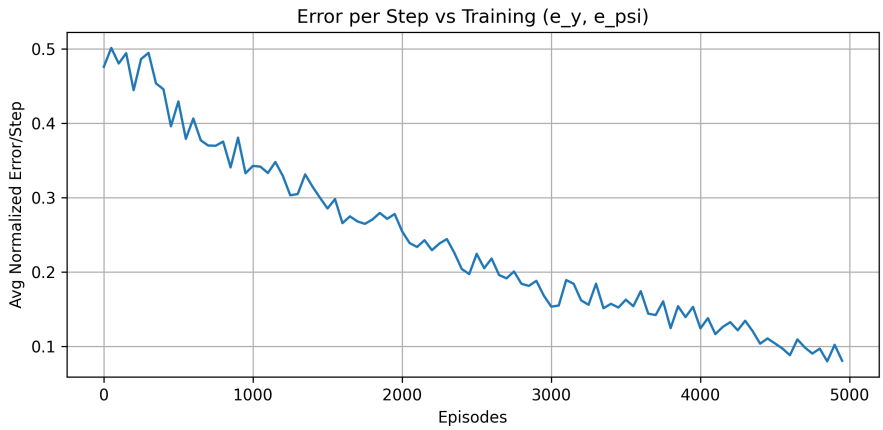


Figure 1: Error per step during training for the basic RL agent.

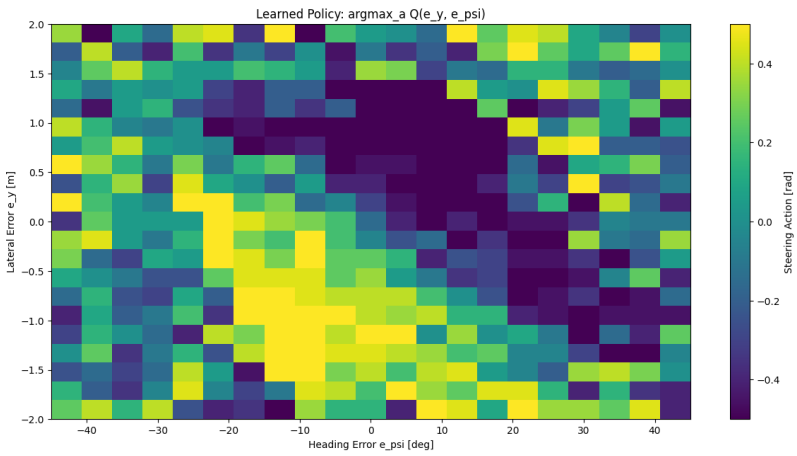


Figure 2: Learned policy heatmap for the basic RL agent (steering as action).

The error-per-step curve (Figure 1) shows a clear decreasing trend, indicating that the agent learns to reduce tracking error over time. The final error is significantly lower than at initialization, suggesting that the agent learns to remain relatively close to the desired path.

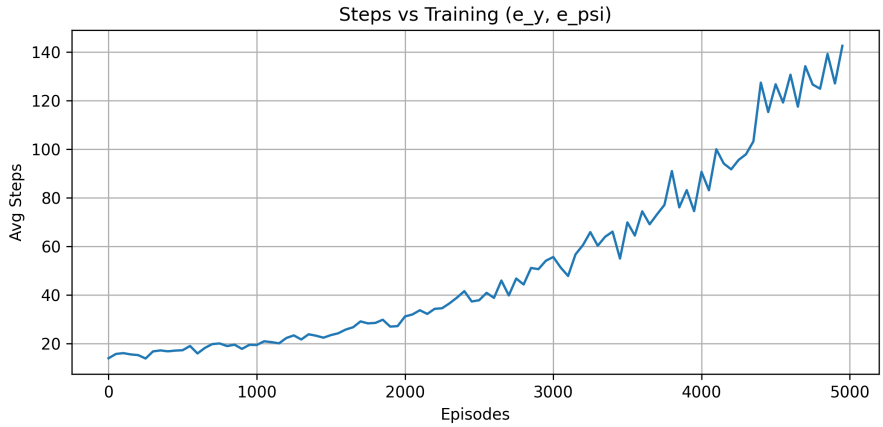


Figure 3: Steps per episode during training for the basic RL agent.

However, the learned policy heatmap (Figure 2) reveals structural weaknesses. The steering map exhibits abrupt transitions between left and right corrective actions. The decision boundary between steering directions is irregular and discontinuous, indicating that the agent reacts locally to error rather than learning a smooth global control strategy. This manifests as visible oscillatory behavior (waviness) and corner cutting.

The steps-per-episode curve (Figure 3) increases during training but remains limited. While survival improves compared to early training, performance plateaus relatively early. This suggests that although tracking error is reduced, long-horizon stability remains suboptimal.

Thus, the basic formulation achieves reasonable tracking accuracy but fails to produce a smooth and dynamically coherent control policy.

4.2 Physics-Informed RL Agent

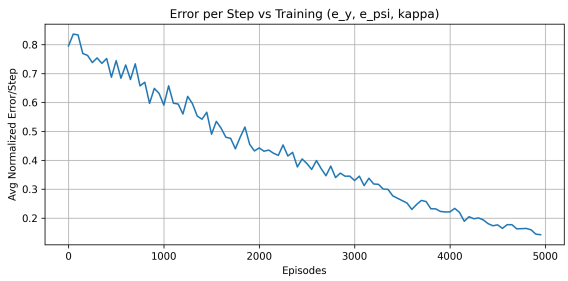
Similarly, the figures for the physics-informed RL agent are shown below.

The error-per-step curve (Figure 4a) again shows steady reduction during training. Importantly, the final error remains comparable to that of the basic agent. This indicates that the inclusion of curvature information does not significantly alter local tracking accuracy.

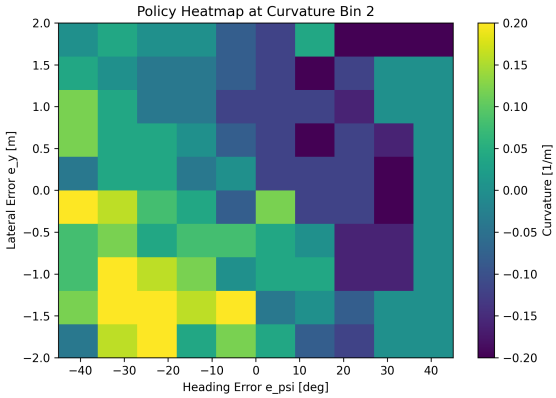
The policy heatmaps across curvature bins (Figures 4b–4d) reveal the most notable improvement. The transitions between left and right corrective actions are smoother and more structured. Instead of irregular discontinuities, steering commands vary more continuously with heading and lateral error. This aligns with the expected behavior of a stabilizing controller.

Across different curvature bins, the learned policy adapts coherently to local path geometry. This suggests that the curvature variable reduces ambiguity in the state representation, enabling the agent to learn a more dynamically consistent control strategy.

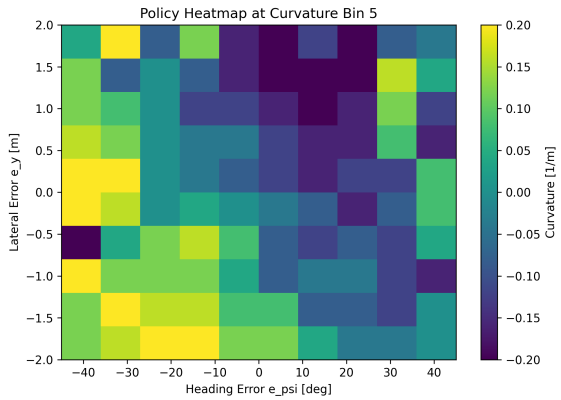
The steps-per-episode curve (Figure 5) shows clear improvement compared to the



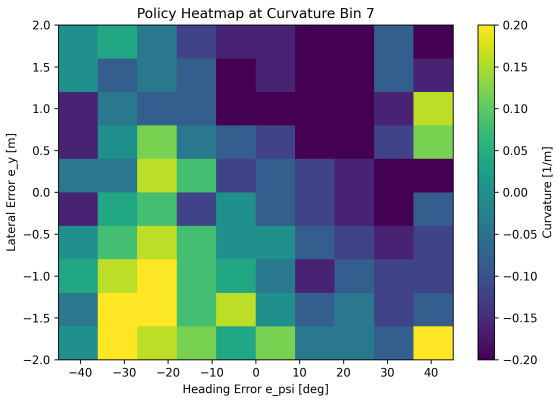
(a) Error per step during training.



(b) Policy heatmap at curvature bin 2.



(c) Policy heatmap at curvature bin 5.



(d) Policy heatmap at curvature bin 7.

Figure 4: Physics-informed RL agent: error curve and learned policies across curvature bins.

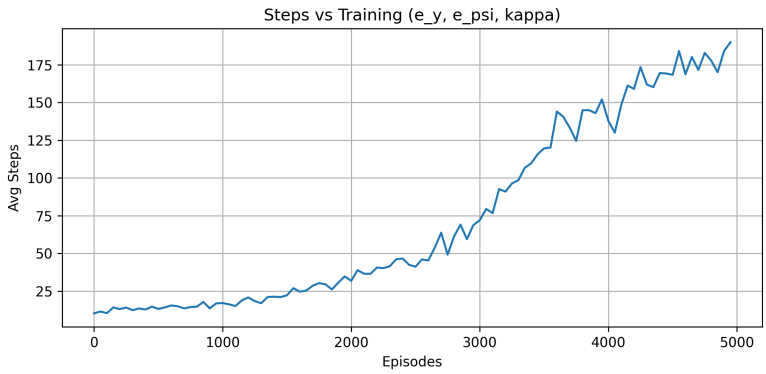


Figure 5: Steps per episode during training for the physics-informed RL agent.

basic agent. Survival increases more substantially and reaches significantly higher values. This indicates improved long-term stability and reduced oscillatory behavior.

Notably, the average tracking error remains similar to the basic formulation. Therefore, the primary improvement lies not in tighter instantaneous tracking, but in smoother control transitions and enhanced temporal coherence.

The inclusion of curvature information appears to regularize the value landscape, leading to smoother policies and improved survival without sacrificing tracking accuracy.

(increasing curvature bins signify that we are going from high curvature counterclockwise to high curvature clockwise)

5 Literature Review

The goal of this section is to review this work and relate it to research already conducted in the fields of control and reinforcement learning. First, we examine work implemented within the Gymnasium environment. Then, we analyze different architectures researchers have used to integrate reinforcement learning into control systems, along with their motivations. Finally, we identify existing gaps and discuss future outlook.

Relating this work to the broader research landscape, what we have done is compare how adding curvature — inspired by how the Pure Pursuit algorithm solves path-following — affects learning performance.

The first group of literature focuses on work done within the Gymnasium environment. Most research branches into two main directions. The first utilizes reinforcement learning as a high-level decision maker. For example, [3] used RL to classify the aggressiveness of different drivers. Similarly, [4] implemented reinforcement learning to learn interaction-aware guidance policies, primarily based on metrics such as time-to-collision (TTC), enabling the system to learn when to overtake and when to platoon.

The second group of work focuses on integrating reinforcement learning on top of existing controllers, and this extends beyond the Gymnasium environment. For example, [5] directly implemented reinforcement learning as a velocity controller while utilizing Pure Pursuit for steering. In this setup, RL was used exclusively for velocity control.

Other approaches aim to integrate RL alongside classical controllers, such as learning PID coefficients online. While similar objectives can be achieved using adaptive control methods (e.g., self-tuning regulators), reinforcement learning provides a data-driven alternative. In [6], RL was used as a standalone controller for nonlinear systems and achieved promising results. In [7], reinforcement learning was implemented on top of a PID controller to adapt its coefficients under varying environmental conditions. This work has also been extended to applications such as inverted pendulum control using an RL-based PID controller.

Returning specifically to path-following, several architectures have incorporated adaptation mechanisms. In [8], an adaptive lookahead distance was integrated into the Pure

Pursuit algorithm, where high curvature results in a smaller lookahead distance and low curvature results in a larger lookahead distance. In [9], reinforcement learning was integrated into an adaptive lateral controller for heavy vehicles on curved roads, achieving promising results.

However, a main gap we have identified is that the literature has not thoroughly addressed how path-following performance varies under changing environmental conditions and unaccounted nonlinearities. A key question that remains is whether reinforcement learning can be used to adapt and improve path-following algorithms when environmental conditions, such as road slipperiness (friction), change suddenly.

6 Conclusion and Future Work

In this report, we examined how different MDP formulations influence learning performance in a path-following task. By adding curvature to the observation space and modifying the action space and reward function accordingly, we demonstrated that incorporating geometric structure into the formulation improves policy smoothness and long-term stability.

The motivation for introducing curvature arose from observing reward hacking behavior in the basic formulation, where the agent learned to cut corners instead of stabilizing smoothly on the path. By including the curvature term, we incentivized the agent to return to the track more coherently and reduce oscillatory behavior. We also strengthened the theoretical foundation of this work by outlining the convergence conditions of the Q-learning algorithm and verifying that both formulations satisfy the assumptions required for theoretical convergence.

Nevertheless, adaptation in the current setup is relatively straightforward due to the absence of noise and disturbances. Additionally, vehicle velocity was kept constant throughout this study. A natural extension of this work is to introduce a longitudinal controller, either learned through reinforcement learning or implemented as a residual controller layered on top of a classical design.

Future directions include increasing environmental complexity by varying terrain slipperiness, introducing static and dynamic obstacles, and evaluating generalization. A critical question moving forward is whether reinforcement learning can truly adapt to new scenarios or whether it has merely learned to solve a specific fixed environment.

References

[1] T. Fermi, "Pure Pursuit," *Algorithms for Automated Driving*. Available at: https://thomasfermi.github.io/Algorithms-for-Automated-Driving/Control/PurePursuit.html. Accessed: January 22, 2025.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction.*

[3] M. T. Regehr and A. Ayoub, "An Elementary Proof that Q-learning Converges Almost Surely," arXiv:2108.02827, 2021.

[4] L. Forneris et al., "Implementing Deep Reinforcement Learning (DRL)-based Driving Styles for Non-Player Vehicles," 2023.

[5] B. Brito, A. Agarwal, and J. Alonso-Mora, "Learning Interaction-aware Guidance Policies for Motion Planning in Dense Traffic Scenarios."

[6] Y. Cao, K. Ni, T. Kawaguchi, and S. Hashimoto, "Path Following for Autonomous Mobile Robots with Deep Reinforcement Learning."

[7] H. Hassanpour, P. Mhaskar, and B. Corbett, "A Practical Reinforcement Learning (RL) Controller Design for Nonlinear Systems," American Control Conference, 2024.

[8] Q. Shi, H.-K. Lam, B. Xiao, and S.-H. Tsai, "Adaptive PID Controller Based on Q-learning Algorithm."

[9] R. Al Fata and N. Daher, "Adaptive Pure Pursuit with Deviation Model Regulation for Trajectory Tracking in Small-Scale Racecars," IC2AI, 2025.